The Dark Side of Ethereum 1/64th CALL Gas Reduction

In this article we argue that the 1/64 CALL gas reduction in EIP150 is problematic and we suggest Ethereum 2.0 should revert this change. That's why RSK implemented EIP150, but excluded this rule.

Sergio Demian Lerner

In this post we'll explain what we think was the real objective of the 1/64 gas reduction for CALL/CREATE, first described as the "Option A" in EIP114 and later activated in Ethereum Tangerine Whistle hard fork. We argue that the reasons to merge EIP114 in Ethereum were not correctly communicated to the community. The 1/64 rule brings problems to the Ethereum 1.0 platform, and the problems could aggravate in Ethereum 2.0. we'll also show why RSK did not implement EIP114 and present alternative solutions.

The History of the 1/64 rule

EIP114 was written by Vitalik in 2016. To understand why Ethereum adopted EIP114 we must first discover the original motivation. EIP114 seems to have a connection with EIP90, but that's only superficial. Some developers seem to believe that the reasons EIP114 was adopted are related with the repricing of the CALL opcode. However we think EIP114 hides the most probable reason for the change. A glimpse of this reason for EIP114 is hidden in one sentence of EIP150 itself. Let's review EIP114:

> For blocks where `block.number >= METROPOLIS_HARDFORK_BLKNUM`, make the following changes:
>
> 1. The 1024 call stack limit no longer exists
> 2. Still keep track of the call stack depth; however, if the call stack depth is at least 1024 (ie. in and only in those execution environments which would never be reachable in the current Ethereum implementation because they would trigger the max call stack depth exception), a `CALL`, `CALLCODE`, `CREATE` or `DELEGATECALL` can allocate a maximum of `(g * 63) // 64` gas to the child, where `g` is the remaining gas in the message at the time the call is made, after subtracting gas costs for the call and for memory expansion.
>
> … with this mechanism for enforcing a maximum call stack depth, **contracts no longer have to worry about the remaining call stack depth in the execution environment they are running in, and possible attacks or bugs if the depth is too low, and instead only need**

> **to worry about the single limiting variable of gas**.

(emphasis is mine)

The last comment of Vitalik is interesting: having a single limiting element (gas) is better than having two (gas + call depth). This is completely true. But is this the main reason? We think the main reason for the push of 1/64 rule was hidden until EIP150 was published.

The safety reasons for the 1/64 rule

EIP150 (initially committed in April 2017) includes the 1/64 change to the consensus and points to EIP114. The comment in EIP150 about EIP114 is the following:

> EIP 114 is introduced because, given that we are **making the cost of a call higher and less predictable**, we have an opportunity to do it at no extra cost to currently available guarantees, and so we also achieve the benefit of replacing the call stack depth limit with a "softer" gas-based restriction, thereby eliminating call **stack depth attacks as a class of attack** that contract developers have to worry about and hence increasing contract programming safety. Note that with the given parameters, the de-facto maximum call stack depth is limited to ~340 (down from ~1024), mitigating the harm caused by any further potential **quadratic-complexity DoS attacks that rely on calls**.

(emphasis is mine)

This paragraph argues that the 1/64 reduction increases programming safety because of stack depth attacks, but by the time the EIP was published the Solidity compiler had already adopted the necessary protections for CALLs. EIP150 states that the 1/64 rule makes the CALL cost less predictable. It however does not attempt to measure how unpredictable it may become or how complex for wallets would be to estimate it. A new algorithm to compute the gas estimation was written and merged into geth here a few months before the publication of the EIP. The difficulty of estimating the gas limit of a transaction is a big limiting factor for the adoption of the platform, as we'll discuss in the following section.

The preference of Vitalik for the 1/64 change must be put in context: when EIP114 was created (June 2016) developers still used the Solidity keywords send() and call() in their contracts. The Solidity compiler did not warn about unchecked send() return value, and often contract authors forgot to manually perform the checks. Therefore it was possible to force a contract to silently fail on send() or call() by reducing the available stack size. Therefore the 1/64 was a protection for already deployed contracts against stack-depth attacks. One of the main reasons for the change wasn't to improve the platform for future contracts, but to protect developers from their own past mistakes (which could have been induced by a lack of proper documentation). In fact a research estimated that about 1500 already deployed contracts had that vulnerability. This is one of he warnings added in 2016 to the Solidity documentation after several successful attacks on deployed contracts:

> **Warning:** There are some dangers in using `send`: The transfer fails if the call stack depth is at 1024 (this can always be forced by the caller) and it also fails if the recipient runs out of gas. So in order to make safe Ether transfers, always check the return value of `send`, use `transfer` or even better: Use a pattern where the recipient withdraws the money.

Soon the Solidity compiler was improved to emit warnings if the programmer forgot to check a send() or call() return code.

Version 0.3.6

chriseth released this on Aug 10, 2016 · 12191 commits to develop since this release

Features:

- Formal verification: Take external effects on a contract into account.
- Type Checker: Warning about unused return value of low-level calls and send.

With the warnings and a change in programming habits, the risks of future bugs were reduced considerably. RSK was launched in January 2018, and it probably doesn't have a single relevant contract vulnerable to this attack. Why should RSK carry the burden of a decision made to protect old contracts? Should Ethereum 2.0 ?
If you're writing code in EVM assembly, then checking return codes would be one of many many safety checks you would need to consider: it is just a drop in the ocean. It must be noted that Solidity always checked error conditions of contract method calls. Here is how the latest version of Solidity checks for stack-overflow and reverts in method calls:

```
        CALL
        ISZERO
        DUP1
        ISZERO
        PUSH [tag] 20
        JUMPI
        RETURNDATASIZE
        PUSH 0
        DUP1
        RETURNDATACOPY
        RETURNDATASIZE
        PUSH 0
        REVERT
    tag 20
```

The other important reason is briefly referred to as "quadratic-complexity DoS attacks that rely on calls". There is no explanation of what this is so I tried to analyze the possible problems that can lead to quadratic behaviour and found the following: the state cache is a data structure that many Ethereum implementations use to temporarily store the modifications that need to be performed to the state if a contract call does not revert (or rises a OOG exception). It is used by Parity (now OpenEthereum) (storage and accounts), Pantheon (storage and accounts) and Ethereumj (storage and accounts). The original Ethereum design yellow paper did not take into account nor suggested which data structure should be used for this cache. It was left to the implementer to decide. But not all data structures can match the designed gas costs. If a contract performs 1000 nested CALLs,

then a single BALANCE query must traverse all 1000 caches until reaching the original state trie, because any of the intermediate call frames could have modified account balances. We can assume that a cache lookup costs approximately 20 gas units because BLOCKHASH, which accesses an in-memory array, costs 20. Then a BALANCE query that travels through 1000 caches should have cost 20K gas instead of 400. The same happens for SLOAD, EXTCODESIZE, and EXTCODEHASH opcodes, but BALANCE and EXTCODEHASH have lower costs. Even if the cost of accessing the cache is low, clearly it wasn't considered in the design of Ethereum or in some implementations. For example, to perform a BALANCE operation at stack depth 1024, execution has to reach depth 1024 with at least 400 gas left. Before EIP150, BALANCE cost was 20. If a transaction gas limit was 8M gas, it could reach depth 1024 consuming 750K gas, and use the remaining gas to perform about 300K pseudo-random BALANCE calls, which results in more than 300M internal dictionary lookups! After EIP150, with the same amount of gas you can perform a single BALANCE at a depth of 600 approximately. A more efficient attack would be to perform 7000 BALANCE queries at depth 66, resulting in 462K cache queries, possibly taking no more than a few hundred milliseconds. You can see that the same attack before EIP150 forces 665 times more process. Assuming standard SSD access times, if each BALANCE operation ends up in two SSD accesses, then that transaction would take 1400 milliseconds to execute, which is high, but it's not a realistic attack.

Geth is the only implementation that avoids this extra lookup cost using a journal instead of a cache. For every change made to the state, the journal stores the previous value and the key in an in-memory dictionary. The new value is directly written to the global state. If the same storage cell is modified several times within the same contract call frame, only the first journal entry needs to persist. The only downside of using a journal is that reversion (REVERT or OOG) may result in a non-constant number of operations. However, because each operation can only be reversed only once, we can assume this cost has already been prepaid by state-modifying opcodes, such as SSTORE, CALLs with value and CREATEs. A key/value reversal, which is performed always in-memory and does not require SSD access, should cost about 40 gas, doubling the cost of a BLOCKHASH opcode. This represents, in the worst case, 10% of the cost of BALANCE, 0.8% of the cost of SSTORE, 1.3% of the cost of CALL with value, and 0.1% of the cost of CREATE.

We conclude that the 1/64 change main objective was to increase the locked gas in nested calls, so that the gas to perform state queries runs out quickly. Was the 1/64 rule a good solution?

Gas estimation is important because Ethereum throws the OOG exception too easily in error situations, consuming all transaction gas, which puts the user at a higher risk than strictly necessary. If transactions executed REVERT on errors, maybe users would be willing to set the transaction gas limit equal to the block gas limit, and forget about it. Gas estimation wouldn't be a problem. When designing RSK we considered changing that behaviour, but maintaining compatibility was considered too important.

Gas Estimation in Ethereum

Wallets need to estimate the gas limit required to set for the transaction to be correctly executed. For most wallets the estimation is performed by calling a web3 method named estimateGas. The method name is confusing, because this method does not estimate the amount of gas consumed, but returns the amount of gas that must be paid up-front for the transaction to execute correctly. The real amount consumed may be a lot lower. Ganache is one exception, because it performs its own estimation. But estimateGas works differently in Ethereum and RSK.
The Ethereum's estimateGas method locally executes the transaction against the state of the best block and returns a viable gas limit with the maximum possible precision. There are at least five obstacles for estimating the gas limit of a transaction:

1. When the transaction is finally included in a block, the state of the system will be different. Called contracts may have changed their internal storage cells and account balances could also be different. Therefore the transaction may fail, consuming all passed gas.
2. Because of the 1/64 rule, a transaction that performs recursive calls will require much more gas paid up-front than the amount it will consume, because each CALL reserves 1/64th of the gas passed. For example, to perform 44 recursive CALLs and execute a contract X in the 44rd level, the Ethereum EVM requires the double amount of the gas paid up-front to be locked by each nested CALL. While the locked gas is returned to the sender, it still needs to be paid up-front. This is especially problematic for the GSN 2.0 that allows the payment of transaction gas in ERC-20 tokens by swapping tokens for ether on a onchain exchange. The unused gas locked in CALLs forces reimbursements by swaping back ether for tokens, and transferring back tokens to the user. The gas overhead of reimbursements and onchain exchange are high.
3. A transaction may remove one or more contracts, if they execute the SELFDESTRUCT opcode. Each destroyed contract refunds 24000 gas. A transaction may require to pay up-front almost twice the gas consumed.
4. Every storage cell that is freed by storing the value zero pays 5000 in gas in advance, but refunds 15000 gas. Again, a transaction may require an up-front payment twice as necessary.
5. A CALL that transfers value requires passing 2300 gas to the callee in advance, even if this gas won't be consumed.

SELFDESTRUCT and SSTORE refunds (3 and 4) are capped to 50% of the spent gas, and therefore the gas limit has an upper bound that depends on the gas spent. The CALL stipend (5) is similar to the 1/64 rule, but since it's constant, it's easier to calculate. Since the same gas amount can be passed from parent to child, and it's not locked at every call frame, the limit becomes the previous gas consumed plus 2300. But the 1/64 rule (2) is much more difficult to compute and could raise the transaction gas limit to over 100 times more than the gas consumed, in the worst case.

Most Ethereum full nodes perform a binary search to find the minimum gas limit that executes the transaction without reaching a OOG exception. This involves executing the contract multiple times. It requires two assumptions on the bounds: the transaction will successfully execute with the maximum possible transaction gas limit, and that it will finish with OOG when the gas limit is set below 21K. You can see how this binary search is performed here and here. Because an Ethereum transaction can currently consume up to 12.5 million gas units, in the average case the program will be executed log2(12M) times, which is 23 times. In 2019 I tried to find the worst case for estimateGas, and I created a specific contract using the worst opcodes in terms of time/gas that forces estimateGas to run for 38 seconds in my standard PC. This varies between different nodes (geth, OpenEthereum, etc.) and nodes have evolved but I think this has not changed. I'm assuming the user can be tricked into estimating the gas for a contract created by an attacker. In the Ethereum ecosystem, most users currently rely on centralized Infura nodes for gas estimation. Infura nodes provide this service for free, but they probably had a time-up of a few seconds before aborting. However Infura nodes will need to handle higher and higher loads for gas estimation, because the block gas limit has been steadily rising.

It is possible but cumbersome to compute the gas limit based on the number of nested calls and the gas passed to each, therefore performing a single execution. It seems that Ganache has solved this analytically here, However this requires instrumenting the VM with special code to extract this additional information, and no Ethereum full node does it.

Stateless nodes of Ethereum 2.0 running the EVM runtime will have a hard time estimating gas cost in the worst case. This is because the EVM can query the current gas with the GAS opcode, and can generate a completely different set of pseudo-random addresses to query balance with the BALANCE opcode on each call in the binary search. Therefore the node would need to fetch data from archive nodes from sparsely generated addresses, and return long Merkle membership proofs for each. In the end, stateless nodes will need to rely on centralized nodes for worst case gas estimation.

Gas Estimation in RSK

Since RSK does not implement the 1/64 reduction, gas estimation in RSK can be much simpler. However currently rskj's estimateGas is imperfect, as it returns the gas consumed rather than the gas limit. However, fixing it is easy, and a pull request to fix it is waiting for approval. It requires instrumenting the VM with minimal code to extract one additional bit of information. When there is a top level call that consumes less than 2300 gas and does transfer value, we add 2300 gas to the gas limit estimation. This is because, apart from the 9000 gas units consumed when a call transfers value, the call is forced to pass at least 2300 gas units to the calle. In addition, the gas refunds for SSTORE and SUICIDE executions must be either skipped, or re-added by the estimation code. The simplicity of RSK gas estimation means that in the future stateless clients can easily compute transaction gas costs, and the network can remain decentralized.

Gas Limit Estimation in The Future

While Ethereum plans to turn its beacon chain in a stateless system to prevent state growth RSK plans to implement storage rent to handle state more efficiently. Storage rent does not need SSTORE/SUICIDE refunds, because the rent system could automatically remove from low latency storage information infrequently accessed. It could either move unused data to a cheaper and slower storage device or remove it completely (a technique called hibernation). Transactions with SSTORE/SELFDESTRUCT refunds may be problematic to users at the current high price of gas, because they may need to pre-pay almost 100% more than the actual consumed amount. Finally, although miners select transactions by higher gas price, transactions with high refunds may be problematic to miners since they can consume twice the CPU gas, and therefore they can delay block generation. Finally, refunds have shown to be useless for the reduction of state size, because few users actively generate contracts that can be cleaned nor transactions with the sole objective of freeing data cells. Newer contracts rarely provide a selfDestruct() method. This is because large refunds can only be monetized through gas tokens with low efficiency. Gas arbitration (acquiring empty cells at low cost to sell them at a higher cost later) has been growing since gas prices in Ethereum skyrocketed. However, gas arbitration is prejudicial to users when used for speculation. Therefore **RSK should remove gas refunds in the future** and gas prediction will be even simpler.

This is aligned with the strategic objectives laid on RSK white paper. The main objective being enabling financial inclusion, which requires simplicity, scalability and safety. Compatibility with Ethereum is not RSK's main objective. Compatibility simplifies the life of developers, but clearly it won't last forever, since both platforms will diverge when Ethereum 2.0 is launched.
Many Ethereum tests depend heavily on the amount of gas consumed (for example, they execute until OOG). A test that is based on gas consumption, rather than on known and planned data transformations, should not be regarded as a good functional test. EVM compatibility should not be measured by opcode gas costs.

How RSK handles EVM CALLs

An EVM CALL in RSK has a basic cost of 700 (as post EIP-150) but does not restrict the gas passed to callee by 1/64th of the available amount. To reduce the impact of quadratic-complexity attacks to the state cache, it limits the stack depth to 400. Because RSK unitrie data structure stores accounts very efficiently in memory, and because RSK contains a much lower number of accounts than Ethereum, this class of attacks are not currently a problem in RSK. However it would be prudent to change the internal state cache data structure to a journal, as in geth. But introducing a journal is a major change and it would require thorough testing and review.

Preventing Stack Depth Attacks in Meta-transaction Systems

In a meta-transaction system, a user and a relayer can settle a dispute on whether a transaction was correctly executed or not using an on-chain contract. For this to happen, both parties must agree on what is “correctly executed”. Generally they will agree on the call destination address, the call arguments and the gas limit that will be used for the call. In Ethereum, that’s enough, but in RSK, they may have to agree on the maximum call stack free space that the destination contract will expect to work correctly. This is generally enforced by both parties trusting a contract that can only be called externally (checking msg-sender==msg.origin), so the call depth is fixed and known. It would be advantageous in some cases that the instruction set contained a STACKDEPTH opcode, or a precompiled contract existed that could return such value.

Summary

The 1/64 rule in EIP150 currently serves two purposes: increases the safety of EVM programs written in assembly, and solves an implementation problem (cache vs journal) at the expense of increasing the complexity of the system in EVM CALLs, and increasing an order of magnitude the cost of gas estimation by wallets and nodes. Also it increases the dependency of light nodes and stateless clients on centralized infura-like servers. Contrary to what is believed, it does not improve security if contracts are written with the latest version of Solidity.

RSK did not implement the 1/64 rule, and therefore it can benefit from an efficient gas estimator (known as exactimator). RSK can in the future improve its cache data structure to prevent quadratic-complexity attacks. We argue that compatibility with Ethereum should focus on the Solidity and web3 interface, which enable easily porting Ethereum dApps to RSK, but not necessarily opcode gas costs. We think that simpler gas estimation will pay back in the upcoming years. Finally we think Ethereum 2.0 should revert the 1/64 rule and remove gas refunds from the EVM design. This will help for the long term continuity of the platform.

Special thanks to Shreemoy Mishra for the useful suggestions